REST Resource: users.dataTypes.dataPoints

Resource: DataPoint

A computed or recorded metric.

JSON representation

```
{
  "name": string,
  "dataSource": {
    object (DataSource (/health/reference/rest/v4/users.dataTypes.dataPoints#DataSource
  },

  // Union field data can be only one of the following:
  "steps": {
    object (Steps (/health/reference/rest/v4/users.dataTypes.dataPoints#Steps))
  },
  "floors": {
    object (Floors (/health/reference/rest/v4/users.dataTypes.dataPoints#Floors))
  },
  "heartRate": {
    object (HeartRate (/health/reference/rest/v4/users.dataTypes.dataPoints#HeartRate))
  },
  "sleep": {
    object (Sleep (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep))
  },
  "dailyRestingHeartRate": {
    object (DailyRestingHeartRate (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "dailyHeartRateVariability": {
    object (DailyHeartRateVariability (/health/reference/rest/v4/users.dataTypes.dat
  },
  "exercise": {
    object (Exercise (/health/reference/rest/v4/users.dataTypes.dataPoints#Exercise))
  },
  "weight": {
    object (Weight (/health/reference/rest/v4/users.dataTypes.dataPoints#Weight))
  },
```

```
  "altitude": {
    object (Altitude (/health/reference/rest/v4/users.dataTypes.dataPoints#Altitude))
  },
  "distance": {
    object (Distance (/health/reference/rest/v4/users.dataTypes.dataPoints#Distance))
  },
  "bodyFat": {
    object (BodyFat (/health/reference/rest/v4/users.dataTypes.dataPoints#BodyFat))
  },
  "activeZoneMinutes": {
    object (ActiveZoneMinutes (/health/reference/rest/v4/users.dataTypes.dataPoints#A
  },
  "heartRateVariability": {
    object (HeartRateVariability (/health/reference/rest/v4/users.dataTypes.dataPoint
  },
  "dailySleepTemperatureDerivations": {
    object (DailySleepTemperatureDerivations (/health/reference/rest/v4/users.data
  },
  "sedentaryPeriod": {
    object (SedentaryPeriod (/health/reference/rest/v4/users.dataTypes.dataPoints#Sede
  },
  "runVo2Max": {
    object (RunVO2Max (/health/reference/rest/v4/users.dataTypes.dataPoints#RunVO2Max
  },
  "oxygenSaturation": {
    object (OxygenSaturation (/health/reference/rest/v4/users.dataTypes.dataPoints#Oxy
  },
  "dailyOxygenSaturation": {
    object (DailyOxygenSaturation (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "activityLevel": {
    object (ActivityLevel (/health/reference/rest/v4/users.dataTypes.dataPoints#Activity
  },
  "vo2Max": {
    object (VO2Max (/health/reference/rest/v4/users.dataTypes.dataPoints#VO2Max))
  },
  "dailyVo2Max": {
    object (DailyVO2Max (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyVO2M
  },
  "nutritionLog": {
    object (NutritionLog (/health/reference/rest/v4/users.dataTypes.dataPoints#Nutrition
  },
  "irregularRhythmNotification": {
    object (IrregularRhythmNotification (/health/reference/rest/v4/users.dataTypes.d
  },
```

```
  "electrocardiogram": {
    object (Electrocardiogram (/health/reference/rest/v4/users.dataTypes.dataPoints#El
  },
  "dailyHeartRateZones": {
    object (DailyHeartRateZones (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "hydrationLog": {
    object (HydrationLog (/health/reference/rest/v4/users.dataTypes.dataPoints#Hydratio
  },
  "food": {
    object (Food (/health/reference/rest/v4/users.dataTypes.dataPoints#Food))
  },
  "timeInHeartRateZone": {
    object (TimeInHeartRateZone (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "activeMinutes": {
    object (ActiveMinutes (/health/reference/rest/v4/users.dataTypes.dataPoints#ActiveM
  },
  "respiratoryRateSleepSummary": {
    object (RespiratoryRateSleepSummary (/health/reference/rest/v4/users.dataTypes.d
  },
  "dailyRespiratoryRate": {
    object (DailyRespiratoryRate (/health/reference/rest/v4/users.dataTypes.dataPoint
  },
  "swimLengthsData": {
    object (SwimLengthsData (/health/reference/rest/v4/users.dataTypes.dataPoints#Swir
  },
  "height": {
    object (Height (/health/reference/rest/v4/users.dataTypes.dataPoints#Height))
  },
  "coreBodyTemperature": {
    object (CoreBodyTemperature (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "activeEnergyBurned": {
    object (ActiveEnergyBurned (/health/reference/rest/v4/users.dataTypes.dataPoints#A
  },
  "foodMeasurementUnit": {
    object (FoodMeasurementUnit (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "bloodGlucose": {
    object (BloodGlucose (/health/reference/rest/v4/users.dataTypes.dataPoints#BloodGlu
  }
  // End of list of possible types for union field data.
}
```

Fields

`name`	`string` Identifier. Data point name, only supported for the subset of identifiable data types. For the majority of the data types, individual data points do not need to be identified and this field would be empty. Format: `users/{user}/dataTypes/{dataType}/dataPoints/{dataPoint}` Example: `users/abcd1234/dataTypes/sleep/dataPoints/a1b2c3d4-e5f6-7890-1234-567890abcdef` The `{user}` ID is a system-generated identifier, as described in `Identity.health_user_id` (/health/reference/rest/v4/users/getIdentity#body.Identity.FIELDS.health_user_id). The `{dataType}` ID corresponds to the kebab-case version of the field names in the `DataPoint data` union field (/health/reference/rest/v4/users.dataTypes.dataPoints#DataPoint), e.g. `total-calories` for the `totalCalories` field. The `{dataPoint}` ID can be client-provided or system-generated. If client-provided, it must be a string of 4-63 characters, containing only lowercase letters, numbers, and hyphens.
`dataSource`	`object (DataSource` (/health/reference/rest/v4/users.dataTypes.dataPoints#DataSource)`)` Optional. Data source information for the metric

Union field `data`. Data type component in the name field will always correspond to the data field set. These messages represent the data payload for each data type and are embedded within the `DataPoint` resource. They are not standalone resources themselves. Unless otherwise noted, data types will exclude data points that are identified as recorded by wearables in intervals when they were not actually worn. `data` can be only one of the following:

`steps`	`object (Steps` (/health/reference/rest/v4/users.dataTypes.dataPoints#Steps)`)` Optional. Data for points in the `steps` interval data type collection.
`floors`	`object (Floors`

	(/health/reference/rest/v4/users.dataTypes.dataPoints#Floors)) Optional. Data for points in the **floors** interval data type collection.
heartRate	object (HeartRate (/health/reference/rest/v4/users.dataTypes.dataPoints#HeartRate)) Optional. Data for points in the **heart-rate** sample data type collection.
sleep	object (Sleep (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep)) Optional. Data for points in the **sleep** session data type collection.
dailyRestingHeartRate	object (DailyRestingHeartRate (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyRestingHeartRate)) Optional. Data for points in the **daily-resting-heart-rate** daily data type collection.
dailyHeartRateVariability	object (DailyHeartRateVariability (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyHeartRateVariability)) Optional. Data for points in the **daily-heart-rate-variability** daily data type collection.
exercise	object (Exercise (/health/reference/rest/v4/users.dataTypes.dataPoints#Exercise)) Optional. Data for points in the **exercise** session data type collection.
weight	object (Weight (/health/reference/rest/v4/users.dataTypes.dataPoints#Weight)) Optional. Data for points in the **weight** sample data type collection.
altitude	object (Altitude (/health/reference/rest/v4/users.dataTypes.dataPoints#Altitude)) Optional. Data for points in the **altitude** interval data type collection.
distance	object (Distance (/health/reference/rest/v4/users.dataTypes.dataPoints#Distance)) Optional. Data for points in the **distance** interval data type collection.

`bodyFat`	`object (`BodyFat (/health/reference/rest/v4/users.dataTypes.dataPoints#BodyFat)`)` Optional. Data for points in the `body-fat` sample data type collection.
`activeZoneMinutes`	`object (`ActiveZoneMinutes (/health/reference/rest/v4/users.dataTypes.dataPoints#ActiveZoneMinutes) `)` Optional. Data for points in the `active-zone-minutes` interval data type collection, measured in minutes.
`heartRate Variability`	`object (`HeartRateVariability (/health/reference/rest/v4/users.dataTypes.dataPoints#HeartRateVariability) `)` Optional. Data for points in the `heart-rate-variability` sample data type collection.
`dailySleep Temperature Derivations`	`object (`DailySleepTemperatureDerivations (/health/reference/rest/v4/users.dataTypes.dataPoints#DailySleepTemperatureDerivations) `)` Optional. Data for points in the `daily-sleep-temperature-derivations` daily data type collection.
`sedentaryPeriod`	`object (`SedentaryPeriod (/health/reference/rest/v4/users.dataTypes.dataPoints#SedentaryPeriod) `)` Optional. Data for points in the `sedentary-period` interval data type collection.
`runVo2Max`	`object (`RunVO2Max (/health/reference/rest/v4/users.dataTypes.dataPoints#RunVO2Max)`)` Optional. Data for points in the `run-vo2-max` sample data type collection.
`oxygenSaturation`	`object (`OxygenSaturation (/health/reference/rest/v4/users.dataTypes.dataPoints#OxygenSaturation) `)`

	Optional. Data for points in the **oxygen-saturation** sample data type collection.
dailyOxygenSaturation	**object (**DailyOxygenSaturation (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyOxygenSaturation)**)** Optional. Data for points in the **daily-oxygen-saturation** daily data type collection.
activityLevel	**object (**ActivityLevel (/health/reference/rest/v4/users.dataTypes.dataPoints#ActivityLevel)**)** Optional. Data for points in the **activity-level** daily data type collection.
vo2Max	**object (**VO2Max (/health/reference/rest/v4/users.dataTypes.dataPoints#VO2Max)**)** Optional. Data for points in the **vo2-max** sample data type collection.
dailyVo2Max	**object (**DailyVO2Max (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyVO2Max)**)** Optional. Data for points in the **daily-vo2-max** daily data type collection.
nutritionLog	**object (**NutritionLog (/health/reference/rest/v4/users.dataTypes.dataPoints#NutritionLog)**)** Optional. Data for points in the **nutrition-log** session data type collection.
irregularRhythmNotification	**object (**IrregularRhythmNotification (/health/reference/rest/v4/users.dataTypes.dataPoints#IrregularRhythmNotification)**)** Optional. Data for points in the **irregular-rhythm-notification** session data type collection.
electrocardiogram	**object (**Electrocardiogram (/health/reference/rest/v4/users.dataTypes.dataPoints#Electrocardiogram)**)** Optional. Data for points in the **electrocardiogram** session data type

	collection.
`dailyHeartRateZones`	`object (`DailyHeartRateZones (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyHeartRateZones)`)` Optional. Data for points in the `daily-heart-rate-zones` daily data type collection.
`hydrationLog`	`object (`HydrationLog (/health/reference/rest/v4/users.dataTypes.dataPoints#HydrationLog)`)` Optional. Data for points in the `hydration-log` session data type collection.
`food`	`object (`Food (/health/reference/rest/v4/users.dataTypes.dataPoints#Food)`)` Optional. The food details.
`timeInHeartRateZone`	`object (`TimeInHeartRateZone (/health/reference/rest/v4/users.dataTypes.dataPoints#TimeInHeartRateZone)`)` Optional. Data for points in the `time-in-heart-rate-zone` interval data type collection.
`activeMinutes`	`object (`ActiveMinutes (/health/reference/rest/v4/users.dataTypes.dataPoints#ActiveMinutes)`)` Optional. Data for points in the `active-minutes` interval data type collection.
`respiratoryRateSleepSummary`	`object (`RespiratoryRateSleepSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#RespiratoryRateSleepSummary)`)` Optional. Data for points in the `respiratory-rate-sleep-summary` sample data type collection.
`dailyRespiratoryRate`	`object (`DailyRespiratoryRate (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyRespiratoryRate)`)`

	Optional. Data for points in the **daily-respiratory-rate** daily data type collection.
swimLengthsData	**object (SwimLengthsData** (/health/reference/rest/v4/users.dataTypes.dataPoints#SwimLengthsData) **)** Optional. Data for points in the **swim-lengths-data** interval data type collection.
height	**object (Height** (/health/reference/rest/v4/users.dataTypes.dataPoints#Height)**)** Optional. Data for points in the **height** sample data type collection.
coreBodyTemperature	**object (CoreBodyTemperature** (/health/reference/rest/v4/users.dataTypes.dataPoints#CoreBodyTemperature) **)** Optional. Data for points in the **core-body-temperature** sample data type collection.
activeEnergyBurned	**object (ActiveEnergyBurned** (/health/reference/rest/v4/users.dataTypes.dataPoints#ActiveEnergyBurned) **)** Optional. Data for points in the **active-energy-burned** interval data type collection.
foodMeasurementUnit	**object (FoodMeasurementUnit** (/health/reference/rest/v4/users.dataTypes.dataPoints#FoodMeasurementUnit) **)** Optional. The food measurement unit details.
bloodGlucose	**object (BloodGlucose** (/health/reference/rest/v4/users.dataTypes.dataPoints#BloodGlucose)**)** Optional. Data for points in the **blood-glucose** sample data type collection.

Steps

Step count over the time interval.

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
  "count": string
}
```

Fields

`interval`	`object (ObservationTimeInterval` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval) `)` Required. Observed interval.
`count`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Required. Number of steps in the recorded interval.

ObservationTimeInterval

Represents a time interval of an observed data point.

JSON representation

```
{
  "startTime": string,
  "startUtcOffset": string,
  "endTime": string,
  "endUtcOffset": string,
  "civilStartTime": {
    object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDa
  },
```

```
  "civilEndTime": {
    object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDa
  }
}
```

Fields

`startTime`	`string (Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) format)` Required. Observed interval start time. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`startUtcOffset`	`string (Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) format)` Required. The offset of the user's local time at the start of the observation relative to the Coordinated Universal Time (UTC). A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`endTime`	`string (Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) format)` Required. Observed interval end time. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`endUtcOffset`	`string (Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) format)` Required. The offset of the user's local time at the end of the observation relative to the Coordinated Universal Time (UTC). A duration in seconds with up to nine fractional digits, ending with '`s`'.

	Example: "3.5s".
civilStartTime	object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDateTime)) Output only. Observed interval start time in civil time in the timezone the subject is in at the start of the observed interval
civilEndTime	object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDateTime)) Output only. Observed interval end time in civil time in the timezone the subject is in at the end of the observed interval

CivilDateTime

Civil time representation similar to google.type.DateTime, but ensures that neither the timezone nor the UTC offset can be set to avoid confusion between civil and physical time queries.

JSON representation

```
{
  "date": {
    object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))
  },
  "time": {
    object (TimeOfDay (/health/reference/rest/v4/users.dataTypes.dataPoints#TimeOfDay))
  }
}
```

Fields

date	object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date)) Required. Calendar date.
time	object (TimeOfDay (/health/reference/rest/v4/users.dataTypes.dataPoints#TimeOfDay)) Optional. Time of day. Defaults to the start of the day, at midnight if

	omitted.

Date

Represents a whole or partial calendar date, such as a birthday. The time of day and time zone are either specified elsewhere or are insignificant. The date is relative to the Gregorian Calendar. This can represent one of the following:

- A full date, with non-zero year, month, and day values.
- A month and day, with a zero year (for example, an anniversary).
- A year on its own, with a zero month and a zero day.
- A year and month, with a zero day (for example, a credit card expiration date).

Related types:

- `google.type.TimeOfDay` (/health/reference/rest/v4/users.dataTypes.dataPoints#TimeOfDay)
- `google.type.DateTime`
- `google.protobuf.Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp)

JSON representation

```
{
  "year": integer,
  "month": integer,
  "day": integer
}
```

Fields

`year`	`integer` Year of the date. Must be from 1 to 9999, or 0 to specify a date without a year.
`month`	`integer`

	Month of a year. Must be from 1 to 12, or 0 to specify a year without a month and day.
`day`	`integer` Day of a month. Must be from 1 to 31 and valid for the year and month, or 0 to specify a year by itself or a year and month where the day isn't significant.

TimeOfDay

Represents a time of day. The date and time zone are either not significant or are specified elsewhere. An API may choose to allow leap seconds. Related types are `google.type.Date` (/health/reference/rest/v4/users.dataTypes.dataPoints#Date) and `google.protobuf.Timestamp`.

JSON representation

```
{
  "hours": integer,
  "minutes": integer,
  "seconds": integer,
  "nanos": integer
}
```

Fields

`hours`	`integer` Hours of a day in 24 hour format. Must be greater than or equal to 0 and typically must be less than or equal to 23. An API may choose to allow the value "24:00:00" for scenarios like business closing time.
`minutes`	`integer` Minutes of an hour. Must be greater than or equal to 0 and less than or equal to 59.
`seconds`	`integer` Seconds of a minute. Must be greater than or equal to 0 and typically must be less than or equal to 59. An API may allow the value 60 if it

	allows leap-seconds.
nanos	**integer** Fractions of seconds, in nanoseconds. Must be greater than or equal to 0 and less than or equal to 999,999,999.

Floors

Gained elevation measured in floors over the time interval

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
  "count": string
}
```

Fields

interval	**object (ObservationTimeInterval** (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval) **)** Required. Observed interval
count	**string (int64** (https://developers.google.com/discovery/v1/type-format) **format)** Required. Number of floors in the recorded interval

HeartRate

A heart rate measurement.

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "metadata": {
    object (HeartRate.HeartRateMetadata (/health/reference/rest/v4/users.dataTypes.
  },
  "beatsPerMinute": string
}
```

Fields

`sampleTime`	`object (ObservationSampleTime` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime) `)` Required. Observation time
`metadata`	`object (HeartRate.HeartRateMetadata` (/health/reference/rest/v4/users.dataTypes.dataPoints#HeartRate.HeartRateMetadata) `)` Optional. Metadata about the heart rate sample.
`beatsPerMinute`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Required. The heart rate value in beats per minute.

ObservationSampleTime

Represents a sample time of an observed data point.

JSON representation

```
{
  "physicalTime": string,
  "utcOffset": string,
  "civilTime": {
    object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDa
```

```
  }
}
```

Fields

`physicalTime`	`string (Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) format)` Required. The time of the observation. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`utcOffset`	`string (Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) format)` Required. The offset of the user's local time during the observation relative to the Coordinated Universal Time (UTC). A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`civilTime`	`object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDateTime))` Output only. The civil time in the timezone the subject is in at the time of the observation.

HeartRate.HeartRateMetadata

Heart rate metadata.

JSON representation

```
{
  "motionContext": enum (HeartRate.HeartRateMetadata.MotionContext (/health/
  "sensorLocation": enum (HeartRate.HeartRateMetadata.SensorLocation (/hea
}
```

Fields	
`motionContext`	`enum (`HeartRate.HeartRateMetadata.MotionContext (/health/reference/rest/v4/users.dataTypes.dataPoints#HeartRate.HeartRateMetadata.MotionContext)`)` Optional. Indicates the user's level of activity when the heart rate sample was measured
`sensorLocation`	`enum (`HeartRate.HeartRateMetadata.SensorLocation (/health/reference/rest/v4/users.dataTypes.dataPoints#HeartRate.HeartRateMetadata.SensorLocation)`)` Optional. Indicates the location of the sensor that measured the heart rate.

HeartRate.HeartRateMetadata.MotionContext

The user's level of activity when the heart rate sample was measured.

Enums	
`MOTION_CONTEXT_UNSPECIFIED`	The default value when no data is available.
`ACTIVE`	The user is active.
`SEDENTARY`	The user is inactive.

HeartRate.HeartRateMetadata.SensorLocation

The location of the sensor that measured the heart rate.

Enums	
`SENSOR_LOCATION_UNSPECIFIED`	The default value when no data is available.

CHEST	Chest sensor.
WRIST	Wrist sensor.
FINGER	Finger sensor.
HAND	Hand sensor.
EAR_LOBE	Ear lobe sensor.
FOOT	Foot sensor.

Sleep

A sleep session possibly including stages.

JSON representation

```
{
  "interval": {
    object (SessionTimeInterval (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "type": enum (Sleep.SleepType (/health/reference/rest/v4/users.dataTypes.dataPoints
  "stages": [
    {
      object (Sleep.SleepStage (/health/reference/rest/v4/users.dataTypes.dataPoints#
    }
  ],
  "outOfBedSegments": [
    {
      object (Sleep.OutOfBedSegment (/health/reference/rest/v4/users.dataTypes.dataP
    }
  ],
  "metadata": {
    object (Sleep.SleepMetadata (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "summary": {
    object (Sleep.SleepSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#
  },
  "createTime": string,
  "updateTime": string
}
```

Fields

`interval`	`object (SessionTimeInterval` (/health/reference/rest/v4/users.dataTypes.dataPoints#SessionTimeInterval)) Required. Observed sleep interval.
`type`	`enum (Sleep.SleepType` (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep.SleepType)) Optional. SleepType: classic or stages.
`stages[]`	`object (Sleep.SleepStage` (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep.SleepStage)) Optional. List of non-overlapping contiguous sleep stage segments that cover the sleep period.
`outOfBedSegments[]`	`object (Sleep.OutOfBedSegment` (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep.OutOfBedSegment)) Optional. “Out of bed” segments that can overlap with sleep stages.
`metadata`	`object (Sleep.SleepMetadata` (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep.SleepMetadata)) Optional. Sleep metadata: processing, main, manually edited, stages status.
`summary`	`object (Sleep.SleepSummary` (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep.SleepSummary)) Output only. Sleep summary: metrics and stages summary.
`createTime`	`string (Timestamp`

	(https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) format) Output only. Creation time of this sleep observation. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "2014-10-02T15:01:23Z", "2014-10-02T15:01:23.045123456Z" or "2014-10-02T15:01:23+05:30".
updateTime	string (Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) format) Output only. Last update time of this sleep observation. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "2014-10-02T15:01:23Z", "2014-10-02T15:01:23.045123456Z" or "2014-10-02T15:01:23+05:30".

SessionTimeInterval

Represents a time interval of session data point, which bundles multiple observed metrics together.

JSON representation

```
{
  "startTime": string,
  "startUtcOffset": string,
  "endTime": string,
  "endUtcOffset": string,
  "civilStartTime": {
    object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDa
  },
  "civilEndTime": {
    object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDa
  }
}
```

Fields

`startTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. The start time of the observed session. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`startUtcOffset`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Required. The offset of the user's local time at the start of the session relative to the Coordinated Universal Time (UTC). A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`endTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. The end time of the observed session. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`endUtcOffset`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Required. The offset of the user's local time at the end of the session relative to the Coordinated Universal Time (UTC). A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`civilStartTime`	`object (CivilDateTime` (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDateTime)`)` Output only. Session start time in civil time in the timezone the subject is in at the start of the session.

`civilEndTime`	`object (`CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDateTime)`)` Output only. Session end time in civil time in the timezone the subject is in at the end of the session.

Sleep.SleepType

Sleep type: classic or stages.

Enums

`SLEEP_TYPE_UNSPECIFIED`	Sleep type is unspecified.
`CLASSIC`	Classic sleep is a sleep with 3 stages types: AWAKE, RESTLESS and ASLEEP.
`STAGES`	On top of "classic" sleep stages an additional processing pass can calculate stages more precisely, overwriting the prior stages with AWAKE, LIGHT, REM and DEEP.

Sleep.SleepStage

Sleep stage segment.

JSON representation

```
{
  "startTime": string,
  "startUtcOffset": string,
  "endTime": string,
  "endUtcOffset": string,
  "type": enum (Sleep.SleepStageType (/health/reference/rest/v4/users.dataTypes.data
  "createTime": string,
  "updateTime": string
}
```

Fields

`startTime`	`string (`Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. Sleep stage start time. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`startUtcOffset`	`string (`Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Required. The offset of the user's local time at the start of the sleep stage relative to the Coordinated Universal Time (UTC). A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`endTime`	`string (`Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. Sleep stage end time. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`endUtcOffset`	`string (`Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Required. The offset of the user's local time at the end of the sleep stage relative to the Coordinated Universal Time (UTC). A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`type`	`enum (`Sleep.SleepStageType (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep.SleepStageType) `)`

	Required. Sleep stage type: AWAKE, DEEP, REM, LIGHT etc.
`createTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Output only. Creation time of this sleep stages segment. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`updateTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Output only. Last update time of this sleep stages segment. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".

Sleep.SleepStageType

Sleep stage type: AWAKE, DEEP, REM, LIGHT etc.

Enums	
`SLEEP_STAGE_TYPE_UNSPECIFIED`	The default unset value.
`AWAKE`	Sleep stage AWAKE.
`LIGHT`	Sleep stage LIGHT.
`DEEP`	Sleep stage DEEP.
`REM`	Sleep stage REM.
`ASLEEP`	Sleep stage ASLEEP.
`RESTLESS`	Sleep stage RESTLESS.

Sleep.OutOfBedSegment

A time interval to represent an out-of-bed segment.

JSON representation

```
{
  "startTime": string,
  "startUtcOffset": string,
  "endTime": string,
  "endUtcOffset": string
}
```

Fields

`startTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. Segment tart time. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`startUtcOffset`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Required. The offset of the user's local time at the start of the segment relative to the Coordinated Universal Time (UTC). A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`endTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. Segment end time. Uses RFC 3339, where generated output will always be Z-normalized and

	use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`endUtcOffset`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Required. The offset of the user's local time at the end of the segment relative to the Coordinated Universal Time (UTC). A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".

Sleep.SleepMetadata

Additional information about how the sleep was processed.

JSON representation

```
{
  "stagesStatus": enum (Sleep.SleepMetadata.StagesState (/health/reference/rest
  "processed": boolean,
  "nap": boolean,
  "manuallyEdited": boolean,
  "externalId": string
}
```

Fields

`stagesStatus`	`enum (Sleep.SleepMetadata.StagesState` (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep.SleepMetadata.StagesState) `)` Output only. Sleep stages algorithm processing status.
`processed`	`boolean` Output only. Sleep and sleep stages algorithms finished processing.
`nap`	`boolean`

	Output only. Naps are sleeps without stages and relatively short durations.
`manuallyEdited`	`boolean` Output only. Some sleeps autodetected by algorithms can be manually edited by users.
`externalId`	`string` Optional. Sleep identifier relevant in the context of the data source.

Sleep.SleepMetadata.StagesState

Sleep stages algorithm processing status.

Enums	
`STAGES_STATE_UNSPECIFIED`	Output only. Sleep stages status is unspecified.
`REJECTED_COVERAGE`	Output only. Sleep stages cannot be computed due to low RR coverage.
`REJECTED_MAX_GAP`	Output only. Sleep stages cannot be computed due to the large middle gap (2h).
`REJECTED_START_GAP`	Output only. Sleep stages cannot be computed due to the large start gap (1h).
`REJECTED_END_GAP`	Output only. Sleep stages cannot be computed due to the large end gap (1h).
`REJECTED_NAP`	Output only. Sleep stages cannot be computed because the sleep log is a nap (has < 3h duration).
`REJECTED_SERVER`	Output only. Sleep stages cannot be computed because input data is not available (PPGV2, wake magnitude, etc).
`TIMEOUT`	Output only. Sleep stages cannot be computed due to server timeout.
`SUCCEEDED`	Output only. Sleep stages successfully computed.
`PROCESSING_INTERNAL_ERROR`	Output only. Sleep stages cannot be computed due to server internal error.

Sleep.SleepSummary

Sleep summary: metrics and stages summary.

JSON representation

```
{
  "stagesSummary": [
    {
      object (Sleep.SleepSummary.StageSummary (/health/reference/rest/v4/users.dat
    }
  ],
  "minutesInSleepPeriod": string,
  "minutesAfterWakeUp": string,
  "minutesToFallAsleep": string,
  "minutesAsleep": string,
  "minutesAwake": string
}
```

Fields

`stagesSummary[]`	`object (Sleep.SleepSummary.StageSummary` (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep.SleepSummary.StageSummary) `)` Output only. List of summaries (total duration and segment count) per each sleep stage type.
`minutesInSleepPeriod`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Output only. Delta between wake time and bedtime. It is the sum of all stages.
`minutesAfterWakeUp`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Output only. Minutes after wake up calculated by restlessness algorithm.
`minutesToFallAsleep`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Output only. Minutes to fall asleep calculated by restlessness algorithm.

`minutesAsleep`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Output only. Total number of minutes asleep. For classic sleep it is the sum of ASLEEP stages (excluding AWAKE and RESTLESS). For "stages" sleep it is the sum of LIGHT, REM and DEEP stages (excluding AWAKE).
`minutesAwake`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Output only. Total number of minutes awake. It is a sum of all AWAKE stages.

Sleep.SleepSummary.StageSummary

Total duration and segment count for a stage.

JSON representation

```
{
  "type": enum (Sleep.SleepStageType (/health/reference/rest/v4/users.dataTypes.data
  "minutes": string,
  "count": string
}
```

Fields

`type`	`enum (Sleep.SleepStageType` (/health/reference/rest/v4/users.dataTypes.dataPoints#Sleep.SleepStageType) `)` Output only. Sleep stage type: AWAKE, DEEP, REM, LIGHT etc.
`minutes`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Output only. Total duration in minutes of a sleep stage.
`count`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Output only. Number of sleep stages segments.

DailyRestingHeartRate

Measures the daily resting heart rate for a user, calculated using the all day heart rate measurements.

JSON representation

```
{
  "date": {
    object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))
  },
  "dailyRestingHeartRateMetadata": {
    object (DailyRestingHeartRate.DailyRestingHeartRateMetadata (/health/refe
  },
  "beatsPerMinute": string
}
```

Fields

date	**object (Date** (/health/reference/rest/v4/users.dataTypes.dataPoints#Date)) Required. Date (in the user's timezone) of the resting heart rate measurement.
dailyRestingHeartRateMetadata	**object (DailyRestingHeartRate.DailyRestingHeartRateMetadata** (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyRestingHeartRate.DailyRestingHeartRateMetadata)) Optional. Metadata for the daily resting heart rate.
beatsPerMinute	**string (int64** (https://developers.google.com/discovery/v1/type-format) **format)** Required. The resting heart rate value in beats per minute.

DailyRestingHeartRate.DailyRestingHeartRateMetadata

Metadata for the daily resting heart rate.

JSON representation

```
{
  "calculationMethod": enum (DailyRestingHeartRate.DailyRestingHeartRateM
}
```

Fields

`calculationMethod`	`enum` (DailyRestingHeartRate.DailyRestingHeartRateMetadata.Ca (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyRestingHeartRate.DailyRestingHeartRateMetadata.CalculationMethod)) Required. The method used to calculate the resting heart rate.

DailyRestingHeartRate.DailyRestingHeartRateMetadata.CalculationMethod

The method used to calculate the resting heart rate.

Enums

`CALCULATION_METHOD_UNSPECIFIED`	The calculation method is unspecified.
`WITH_SLEEP`	The resting heart rate is calculated using the sleep data.
`ONLY_WITH_AWAKE_DATA`	The resting heart rate is calculated using only awake data.

DailyHeartRateVariability

Represents the daily heart rate variability data type.

At least one of the following fields must be set:

- `averageHeartRateVariabilityMilliseconds`
- `nonRemHeartRateBeatsPerMinute`
- `entropy`
- `deepSleepRootMeanSquareOfSuccessiveDifferencesMilliseconds`

JSON representation

```
{
  "date": {
    object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))
  },
  "averageHeartRateVariabilityMilliseconds": number,
  "nonRemHeartRateBeatsPerMinute": string,
  "entropy": number,
  "deepSleepRootMeanSquareOfSuccessiveDifferencesMilliseconds": number
}
```

Fields

`date`	`object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))` Required. Date (in the user's timezone) of heart rate variability measurement.
`averageHeartRateVariabilityMilliseconds`	`number` Optional. A user's average heart rate variability calculated using the root mean square of successive differences (RMSSD) in times between heartbeats.
`nonRemHeartRateBeatsPerMinute`	`string (int64 (https://developers.google.com/discovery/v1/type-format) format)` Optional. Non-REM heart rate
`entropy`	`number` Optional. The Shanon entropy of heartbeat intervals. Entropy quantifies randomness or disorder in a system. High entropy indicates high HRV. Entropy is measured from the histogram of time interval between

	successive heart beats values measured during sleep.
deepSleepRootMeanSquareOfSuccessiveDifferencesMilliseconds `number`	Optional. The root mean square of successive differences (RMSSD) value during deep sleep.

Exercise

An exercise that stores information about a physical activity.

JSON representation

```
{
  "interval": {
    object (SessionTimeInterval (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "exerciseType": enum (Exercise.ExerciseType (/health/reference/rest/v4/users.dat
  "splits": [
    {
      object (Exercise.SplitSummary (/health/reference/rest/v4/users.dataTypes.dataP
    }
  ],
  "exerciseEvents": [
    {
      object (Exercise.ExerciseEvent (/health/reference/rest/v4/users.dataTypes.data
    }
  ],
  "splitSummaries": [
    {
      object (Exercise.SplitSummary (/health/reference/rest/v4/users.dataTypes.dataP
    }
  ],
  "metricsSummary": {
    object (MetricsSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#Metri
  },
  "exerciseMetadata": {
    object (Exercise.ExerciseMetadata (/health/reference/rest/v4/users.dataTypes.dat
  },
  "displayName": string,
  "activeDuration": string,
  "notes": string,
```

```
  "updateTime": string,
  "createTime": string
}
```

Fields

`interval`	`object (`SessionTimeInterval (/health/reference/rest/v4/users.dataTypes.dataPoints#SessionTimeInterval)`)` Required. Observed exercise interval
`exerciseType`	`enum (`Exercise.ExerciseType (/health/reference/rest/v4/users.dataTypes.dataPoints#Exercise.ExerciseType)`)` Required. The type of activity performed during an exercise.
`splits[]`	`object (`Exercise.SplitSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#Exercise.SplitSummary)`)` Optional. The default split is 1 km or 1 mile. - if the movement distance is less than the default, then there are no splits - if the movement distance is greater than or equal to the default, then we have splits
`exerciseEvents[]`	`object (`Exercise.ExerciseEvent (/health/reference/rest/v4/users.dataTypes.dataPoints#Exercise.ExerciseEvent)`)` Optional. Exercise events that happen during an exercise, such as pause & restarts.
`splitSummaries[]`	`object (`Exercise.SplitSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#Exercise.SplitSummary)`)` Optional. Laps or splits recorded within an exercise. Laps could be split based on distance or other criteria (duration, etc.) Laps should not be overlapping with each other.

`metricsSummary`	`object (`MetricsSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#MetricsSummary)`)` Required. Summary metrics for this exercise ( )
`exerciseMetadata`	`object (`Exercise.ExerciseMetadata (/health/reference/rest/v4/users.dataTypes.dataPoints#Exercise.ExerciseMetadata)`)` Optional. Additional exercise metadata.
`displayName`	`string` Required. Exercise display name.
`activeDuration`	`string (`Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Optional. Duration excluding pauses. A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`notes`	`string` Optional. Standard free-form notes captured at manual logging.
`updateTime`	`string (`Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Output only. This is the timestamp of the last update to the exercise. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`createTime`	`string (`Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Output only. Represents the timestamp of the creation of the exercise. Uses RFC 3339, where generated output will always be Z-normalized and

	use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".

Exercise.ExerciseType

The type of activity performed during an exercise.

Enums	
`EXERCISE_TYPE_UNSPECIFIED`	Exercise type is unspecified.
`RUNNING`	Running type.
`WALKING`	Walking type.
`BIKING`	Biking type.
`SWIMMING`	Swimming type.
`HIKING`	Hiking type.
`YOGA`	Yoga type.
`PILATES`	Pilates type.
`WORKOUT`	Workout type.
`HIIT`	HIIT type.
`WEIGHTLIFTING`	Weightlifting type.
`STRENGTH_TRAINING`	Strength training type.
`OTHER`	Other type.

Exercise.SplitSummary

Represents splits or laps recorded within an exercise. Lap events partition a workout into segments based on criteria like distance, time, or calories.

JSON representation

```
{
  "startTime": string,
  "startUtcOffset": string,
  "endTime": string,
  "endUtcOffset": string,
  "activeDuration": string,
  "metricsSummary": {
    object (MetricsSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#Metric
  },
  "splitType": enum (Exercise.SplitSummary.SplitType (/health/reference/rest/v4/
}
```

Fields

`startTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. Lap start time Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`startUtcOffset`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Required. Lap start time offset from UTC A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`endTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. Lap end time Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-`

	02T15:01:23.045123456Z" or "2014-10-02T15:01:23+05:30".
endUtcOffset	string (Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) format) Required. Lap end time offset from UTC A duration in seconds with up to nine fractional digits, ending with 's'. Example: "3.5s".
activeDuration	string (Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) format) Output only. Lap time excluding the pauses. A duration in seconds with up to nine fractional digits, ending with 's'. Example: "3.5s".
metricsSummary	object (MetricsSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#MetricsSummary)) Required. Summary metrics for this split.
splitType	enum (Exercise.SplitSummary.SplitType (/health/reference/rest/v4/users.dataTypes.dataPoints#Exercise.SplitSummary.SplitType)) Required. Method used to split the exercise laps. Users may manually mark the lap as complete even if the tracking is automatic.

MetricsSummary

Summary metrics for an exercise.

JSON representation

```
{
  "heartRateZoneDurations": {
    object (MetricsSummary.TimeInHeartRateZones (/health/reference/rest/v4/users.d
```

```
  },
  "mobilityMetrics": {
    object (MetricsSummary.MobilityMetrics (/health/reference/rest/v4/users.dataTyp
  },
  "caloriesKcal": number,
  "distanceMillimeters": number,
  "steps": string,
  "averageSpeedMillimetersPerSecond": number,
  "averagePaceSecondsPerMeter": number,
  "averageHeartRateBeatsPerMinute": string,
  "elevationGainMillimeters": number,
  "activeZoneMinutes": string,
  "runVo2Max": number,
  "totalSwimLengths": number
}
```

Fields

heartRateZoneDurations	**object (MetricsSummary.TimeInHeartRateZones** (/health/reference/rest/v4/users.dataTypes.dataPoints#MetricsSummary.TimeInHeartRateZones) **)** Optional. Time spent in each heart rate zone.
mobilityMetrics	**object (MetricsSummary.MobilityMetrics** (/health/reference/rest/v4/users.dataTypes.dataPoints#MetricsSummary.MobilityMetrics) **)** Optional. Mobility workouts specific metrics. Only present in the advanced running exercises.
caloriesKcal	**number** Optional. Total calories burned by the user during the exercise.
distanceMillimeters	**number** Optional. Total distance covered by the user during the exercise.
steps	**string (int64** (https://developers.google.com/discovery/v1/type-format) **format)** Optional. Total steps taken during the exercise.

`averageSpeedMillimetersPerSecond`	`number` Optional. Average speed in millimeters per second.
`averagePaceSecondsPerMeter`	`number` Optional. Average pace in seconds per meter.
`averageHeartRateBeatsPerMinute`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Optional. Average heart rate during the exercise.
`elevationGainMillimeters`	`number` Optional. Total elevation gain during the exercise.
`activeZoneMinutes`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Optional. Total active zone minutes for the exercise.
`runVo2Max`	`number` Optional. Run VO2 max value for the exercise. Only present in the running exercises at the top level as in the summary of the whole exercise.
`totalSwimLengths`	`number` Optional. Number of full pool lengths completed during the exercise. Only present in the swimming exercises at the top level as in the summary of the whole exercise.

MetricsSummary.TimeInHeartRateZones

Time spent in each heart rate zone.

JSON representation

```
{
  "lightTime": string,
  "moderateTime": string,
  "vigorousTime": string,
  "peakTime": string
}
```

Fields

`lightTime`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Optional. Time spent in light heart rate zone. A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`moderateTime`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Optional. Time spent in moderate heart rate zone. A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`vigorousTime`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Optional. Time spent in vigorous heart rate zone. A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`peakTime`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Optional. Time spent in peak heart rate zone. A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".

MetricsSummary.MobilityMetrics

Mobility workouts specific metrics

JSON representation

```
{
  "avgGroundContactTimeDuration": string,
  "avgCadenceStepsPerMinute": number,
  "avgStrideLengthMillimeters": string,
  "avgVerticalOscillationMillimeters": string,
  "avgVerticalRatio": number
}
```


Fields

`avgGroundContactTimeDuration`	`string (`Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Optional. The ground contact time for a particular stride is the amount of time for which the foot was in contact with the ground on that stride A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`avgCadenceStepsPerMinute`	`number` Optional. Cadence is a measure of the frequency of your foot strikes. Steps / min in real time during workout.
`avgStrideLengthMillimeters`	`string (`int64 (https://developers.google.com/discovery/v1/type-format) `format)` Optional. Stride length is a measure of the distance covered by a single stride
`avgVerticalOscillationMillimeters`	`string (`int64 (https://developers.google.com/discovery/v1/type-format) `format)` Optional. Distance off the ground your center of mass moves with each stride while running
`avgVerticalRatio`	`number` Optional. Vertical oscillation/stride length between [5.0, 11.0].

Exercise.SplitSummary.SplitType

The type of the split, such as manual, duration, distance, calories.

Enums

`SPLIT_TYPE_UNSPECIFIED`	Split type is unspecified.
`MANUAL`	Manual split.
`DURATION`	Split by duration.
`DISTANCE`	Split by distance.
`CALORIES`	Split by calories.

Exercise.ExerciseEvent

Represents instantaneous events that happen during an exercise, such as start, stop, pause, split.

JSON representation

```
{
  "eventTime": string,
  "eventUtcOffset": string,
  "exerciseEventType": enum (Exercise.ExerciseEvent.ExerciseEventType (/he
}
```

Fields

`eventTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. Exercise event time Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`eventUtcOffset`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)`

	Required. Exercise event time offset from UTC A duration in seconds with up to nine fractional digits, ending with 's'. Example: `"3.5s"`.
`exerciseEventType`	`enum (`Exercise.ExerciseEvent.ExerciseEventType (/health/reference/rest/v4/users.dataTypes.dataPoints#Exercise.ExerciseEvent.ExerciseEventType)`)` Required. The type of the event, such as start, stop, pause, resume.

Exercise.ExerciseEvent.ExerciseEventType

The type of the event, such as start, stop, pause, resume.

Enums

`EXERCISE_EVENT_TYPE_UNSPECIFIED`	Exercise event type is unspecified.
`START`	Exercise start event.
`STOP`	Exercise stop event.
`PAUSE`	Exercise pause event.
`RESUME`	Exercise resume event.
`AUTO_PAUSE`	Exercise auto-pause event.
`AUTO_RESUME`	Exercise auto-resume event.

Exercise.ExerciseMetadata

Additional exercise metadata.

JSON representation

```
{
  "hasGps": boolean,
```

```
  "poolLengthMillimeters": string
}
```

Fields

`hasGps`	`boolean` Optional. Whether the exercise had GPS tracking.
`poolLength Millimeters`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Optional. Pool length in millimeters. Only present in the swimming exercises.

Weight

Body weight measurement.

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "notes": string,
  "weightGrams": number
}
```

Fields

`sampleTime`	`object (ObservationSampleTime` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime)) Required. The time at which the weight was measured
`notes`	`string` Optional. Standard free-form notes captured at manual logging.

`weightGrams`	`number` Required. Weight of a user in grams.

Altitude

Captures the altitude gain (i.e. deltas), and not level above sea, for a user in millimeters.

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
  "gainMillimeters": string
}
```

Fields

`interval`	`object (ObservationTimeInterval` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval)) Required. Observed interval.
`gainMillimeters`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Required. Altitude gain in millimeters over the observed interval.

Distance

Distance traveled over an interval of time.

JSON representation

```
{
```

```
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
  "millimeters": string
}
```

Fields

interval	**object (ObservationTimeInterval** (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval) **)** Required. Observed interval.
millimeters	**string (int64** (https://developers.google.com/discovery/v1/type-format) **format)** Required. Distance in millimeters over the observed interval.

BodyFat

Body fat measurement.

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "percentage": number
}
```

Fields

sampleTime	**object (ObservationSampleTime** (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime) **)** Required. The time at which body fat was measured.

`percentage`	`number` Required. Body fat percentage, in range [0, 100].

ActiveZoneMinutes

Record of active zone minutes in a given time interval.

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
  "heartRateZone": enum (ActiveZoneMinutes.HeartRateZone (/health/reference/res
  "activeZoneMinutes": string
}
```

Fields

`interval`	`object (ObservationTimeInterval` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval) `)` Required. Observed interval.
`heartRateZone`	`enum (ActiveZoneMinutes.HeartRateZone` (/health/reference/rest/v4/users.dataTypes.dataPoints#ActiveZoneMinutes.HeartRateZone) `)` Required. Heart rate zone in which the active zone minutes have been earned, in the given time interval.
`activeZoneMinutes`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Required. Number of Active Zone Minutes earned in the given time interval. Note: activeZoneMinutes equals to 1 for low intensity (fat burn) zones or 2 for high intensity zones (cardio, peak).

ActiveZoneMinutes.HeartRateZone

Represents different heart rate zones.

Enums

HEART_RATE_ZONE_UNSPECIFIED	Unspecified heart rate zone.
FAT_BURN	The fat burn heart rate zone.
CARDIO	The cardio heart rate zone.
PEAK	The peak heart rate zone.

HeartRateVariability

Captures user's heart rate variability (HRV) as measured by the root mean square of successive differences (RMSSD) between normal heartbeats or by standard deviation of the inter-beat intervals (SDNN).

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "rootMeanSquareOfSuccessiveDifferencesMilliseconds": number,
  "standardDeviationMilliseconds": number
}
```

Fields

sampleTime	**object (ObservationSampleTime** (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime)) Required. The time of the heart rate variability measurement.

`rootMeanSquareOfSuccessiveDifferencesMilliseconds`	`number` Optional. The root mean square of successive differences between normal heartbeats. This is a measure of heart rate variability used by Google Health.
`standardDeviationMilliseconds`	`number` Optional. The standard deviation of the heart rate variability measurement.

DailySleepTemperatureDerivations

Provides derived sleep temperature values, calculated from skin or internal device temperature readings during sleep.

JSON representation

```
{
  "date": {
    object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))
  },
  "nightlyTemperatureCelsius": number,
  "baselineTemperatureCelsius": number,
  "relativeNightlyStddev30dCelsius": number
}
```

Fields

`date`	`object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))` Required. Date for which the sleep temperature derivations are calculated.
`nightlyTemperatureCelsius`	`number` Required. The user's nightly skin temperature. It is the mean of skin temperature samples taken from the user’s sleep.
`baselineTemperatureCelsius`	`number` Optional. The user's baseline skin temperature. It is the median of the

	user's nightly skin temperature over the past 30 days.
`relativeNightlyStddev30dCelsius`	`number` Optional. The standard deviation of the user's relative nightly skin temperature (temperature - baseline) over the past 30 days.

SedentaryPeriod

SedentaryPeriod

SedentaryPeriod data represents the periods of time that the user was sedentary (i.e. not moving while wearing the device).

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  }
}
```

Fields

`interval`	`object (ObservationTimeInterval` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval)) Required. Observed interval.

RunVO2Max

VO2 max value calculated based on the user's running activity. Value stored in ml/kg/min.

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "runVo2Max": number
}
```

Fields

sampleTime	**object (ObservationSampleTime** (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime)) Required. The time at which the metric was measured.
runVo2Max	**number** Required. Run VO2 max value in ml/kg/min.

OxygenSaturation

Captures the user's instantaneous oxygen saturation percentage (SpO2).

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "percentage": number
}
```

Fields

sampleTime	**object (ObservationSampleTime** (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime)) Required. The time at which oxygen saturation was measured.

`percentage`	`number` Required. The oxygen saturation percentage. Valid values are from 0 to 100.

DailyOxygenSaturation

A daily oxygen saturation (SpO2) record. Represents the user's daily oxygen saturation summary, typically calculated during sleep.

JSON representation

```
{
  "date": {
    object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))
  },
  "averagePercentage": number,
  "lowerBoundPercentage": number,
  "upperBoundPercentage": number,
  "standardDeviationPercentage": number
}
```

Fields

`date`	`object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))` Required. Date (in user's timezone) of the daily oxygen saturation record.
`averagePercentage`	`number` Required. The average value of the oxygen saturation samples during the sleep.
`lowerBoundPercentage`	`number` Required. The lower bound of the confidence interval of oxygen saturation samples during sleep.
`upperBoundPercentage`	`number` Required. The upper bound of the confidence interval of oxygen saturation samples during sleep.

`standardDeviationPercentage`	`number` Optional. Standard deviation of the daily oxygen saturation averages from the past 7-30 days.

ActivityLevel

Internal type to capture activity level during a certain time interval.

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
  "activityLevelType": enum (ActivityLevel.ActivityLevelType (/health/referenc
}
```

Fields

`interval`	`object (ObservationTimeInterval` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval)) Required. Observed interval.
`activityLevelType`	`enum (ActivityLevel.ActivityLevelType` (/health/reference/rest/v4/users.dataTypes.dataPoints#ActivityLevel.ActivityLevelType)) Required. Activity level type in the given time interval.

ActivityLevel.ActivityLevelType

Represents different activity level types.

Enums

`ACTIVITY_LEVEL_TYPE_UNSPECIFIED`	Unspecified activity level type.
`SEDENTARY`	Sedentary activity level.
`LIGHTLY_ACTIVE`	Lightly active activity level.
`MODERATELY_ACTIVE`	Moderately active activity level.
`VERY_ACTIVE`	Very active activity level.

VO2Max

VO2 max measurement.

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "measurementMethod": enum (VO2Max.MeasurementMethod (/health/reference/rest/v4
  "vo2Max": number
}
```

Fields

`sampleTime`	`object (ObservationSampleTime` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime) `)` Required. The time at which VO2 max was measured.
`measurementMethod`	`enum (VO2Max.MeasurementMethod` (/health/reference/rest/v4/users.dataTypes.dataPoints#VO2Max.MeasurementMethod) `)` Optional. The method used to measure the VO2 max value.

`vo2Max`	`number` Required. VO2 max value measured as in ml consumed oxygen / kg of body weight / min.

VO2Max.MeasurementMethod

Measurement method used to measure the VO2 max value.

Enums	
`MEASUREMENT_METHOD_UNSPECIFIED`	Unspecified measurement method.
`FITBIT_RUN`	Fitbit specific, measures VO2 max rate during a run.
`GOOGLE_DEMOGRAPHIC`	Google specific, measures VO2 max rate for a user based on their demographic data.
`COOPER_TEST`	Run as far as possible for 12 minutes. Distance correlated with age and gender translates to a VO2 max value.
`HEART_RATE_RATIO`	Maximum heart rate divided by the resting heart rate, with a multiplier applied. Does not require any exercise.
`METABOLIC_CART`	Measured by a medical device called metabolic cart.
`MULTISTAGE_FITNESS_TEST`	Continuous 20m back-and-forth runs with increasing difficulty, until exhaustion.
`ROCKPORT_FITNESS_TEST`	Measured using walking exercise.
`MAX_EXERCISE`	Healthkit specific, measures VO2 max rate by monitoring exercise to the user's physical limit. Similar to COOPER_TEST or MULTISTAGE_FITNESS_TEST.
`PREDICTION_SUB_MAX_EXERCISE`	Healthkit specific, estimates VO2 max rate based on low-intensity exercise. Similar to ROCKPORT_FITNESS_TEST.
`PREDICTION_NON_EXERCISE`	Healthkit specific, estimates VO2 max rate without any exercise. Similar to HEART_RATE_RATIO.
`OTHER`	Use when the method is not covered in this enum.

DailyVO2Max

Contains a daily summary of the user's VO2 max (cardio fitness score), which is the maximum rate of oxygen the body can use during exercise.

JSON representation

```
{
  "date": {
    object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))
  },
  "estimated": boolean,
  "cardioFitnessLevel": enum (DailyVO2Max.CardioFitnessLevel (/health/referenc
  "vo2Max": number,
  "vo2MaxCovariance": number
}
```

Fields

date	**object (Date** (/health/reference/rest/v4/users.dataTypes.dataPoints#Date)**)** Required. The date for which the Daily VO2 max was measured.
estimated	**boolean** Optional. An estimated field is added to indicate when the confidence has decreased sufficiently to consider the value an estimation.
cardioFitnessLevel	**enum (DailyVO2Max.CardioFitnessLevel** (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyVO2Max.CardioFitnessLevel)**)** Optional. Represents the user's cardio fitness level based on their VO2 max.
vo2Max	**number** Required. Daily VO2 max value measured as in ml consumed oxygen / kg of body weight / min.
vo2MaxCovariance	**number** Optional. The covariance of the VO2 max value.

DailyVO2Max.CardioFitnessLevel

The cardio fitness level categories.

Enums

`CARDIO_FITNESS_LEVEL_UNSPECIFIED`	Unspecified cardio fitness level.
`POOR`	Poor cardio fitness level.
`FAIR`	Fair cardio fitness level.
`AVERAGE`	Average cardio fitness level.
`GOOD`	Good cardio fitness level.
`VERY_GOOD`	Very good cardio fitness level.
`EXCELLENT`	Excellent cardio fitness level.

NutritionLog

Holds information about a user logged food.

There are two ways of creating a nutrition log based on the food type: 1. Identified food: Using the food field, which is a reference to a Food resource. In this case fields `nutrients`, `energy`, `energyFromFat`, `totalCarbohydrate`, `totalFat`, `foodDisplayName` will be populated based on the referenced food. 2. Anonymous food: Using the `foodDisplayName` field and setting the `nutrients`, `energy`, `energyFromFat`, `totalCarbohydrate`, `totalFat` fields manually.

The identified food is preferred over the anonymous food. Nutrition logs created from anonymous food are not be editable.

JSON representation

```
{
```

```
  "interval": {
    object (SessionTimeInterval (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "nutrients": [
    {
      object (NutrientQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#
    }
  ],
  "energy": {
    object (EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Energ
  },
  "energyFromFat": {
    object (EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Energ
  },
  "totalCarbohydrate": {
    object (WeightQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Weigh
  },
  "totalFat": {
    object (WeightQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Weigh
  },
  "mealType": enum (MealType (/health/reference/rest/v4/users.dataTypes.dataPoints#Me
  "serving": {
    object (NutritionLog.Serving (/health/reference/rest/v4/users.dataTypes.dataPoint
  },
  "food": string,
  "foodDisplayName": string
}
```

Fields

interval	**object (SessionTimeInterval** (/health/reference/rest/v4/users.dataTypes.dataPoints#SessionTimeInterval) **)** Required. Observed interval.
nutrients[]	**object (NutrientQuantity** (/health/reference/rest/v4/users.dataTypes.dataPoints#NutrientQuantity) **)** Optional. Value representing the nutrients of the nutrition log.
energy	**object (EnergyQuantity**

	(/health/reference/rest/v4/users.dataTypes.dataPoints#EnergyQuantity)) Optional. Value representing the energy of the nutrition log. For nutrition logs created from an identified food, this field will be populated based on the referenced food. For anonymous food, this field will be populated manually.
`energyFromFat`	`object (`EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#EnergyQuantity)) Optional. Value representing the energy from fat of the nutrition log. For nutrition logs created from an identified food, this field will be populated based on the referenced food. For anonymous food, this field will be populated manually.
`totalCarbohydrate`	`object (`WeightQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#WeightQuantity)) Optional. Value representing the total carbohydrate of the nutrition log. For nutrition logs created from an identified food, this field will be populated based on the referenced food. For anonymous food, this field will be populated manually.
`totalFat`	`object (`WeightQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#WeightQuantity)) Optional. Value representing the total fat of the nutrition log. For nutrition logs created from an identified food, this field will be populated based on the referenced food. For anonymous food, this field will be populated manually.
`mealType`	`enum (`MealType (/health/reference/rest/v4/users.dataTypes.dataPoints#MealType)`)` Optional. Value representing the meal type of the nutrition log.
`serving`	`object (`NutritionLog.Serving (/health/reference/rest/v4/users.dataTypes.dataPoints#NutritionLog.Serving)) Optional. Value representing the nutrition log serving.
`food`	`string`

	Required. Represents the food ID.
foodDisplayName	**string** Value representing the display name of the food. For nutrition logs created from an identified food, this field will be populated based on the referenced food. For anonymous food, this field will be populated manually.

NutrientQuantity

Represents the quantity of a nutrient.

JSON representation

```
{
  "quantity": {
    object (WeightQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Weigh
  },
  "nutrient": enum (Nutrient (/health/reference/rest/v4/users.dataTypes.dataPoints#Nu
}
```

Fields

quantity	**object (WeightQuantity** (/health/reference/rest/v4/users.dataTypes.dataPoints#WeightQuantity)**)** Required. Value representing the quantity of the nutrient.
nutrient	**enum (Nutrient** (/health/reference/rest/v4/users.dataTypes.dataPoints#Nutrient)**)** Required. Value representing the nutrient.

WeightQuantity

Represents the weight quantity.

JSON representation

```
{
  "userProvidedUnit": enum (WeightUnit (/health/reference/rest/v4/users.dataTypes.da
  "grams": number
}
```

Fields

`userProvidedUnit`	`enum (WeightUnit (/health/reference/rest/v4/users.dataTypes.dataPoints#WeightUnit))` Optional. Value representing the user provided unit.
`grams`	`number` Required. Value representing the weight in grams.

WeightUnit

Enum representing the unit of weight.

Enums

`WEIGHT_UNIT_UNSPECIFIED`	Unspecified weight unit.
`GRAM`	Value representing gram.
`KILOGRAM`	Value representing kilogram.
`OUNCE`	Value representing ounce.
`POUND`	Value representing pound.
`STONE`	Value representing stone.
`MILLIGRAM`	Value representing milligram.
`MICROGRAM`	Value representing microgram.
`NANOGRAM`	Value representing nanogram.

Nutrient

Holds information about a user logged food.

Enums	
NUTRIENT_UNSPECIFIED	Unspecified nutrient.
BIOTIN	Value representing biotin nutrient.
CAFFEINE	Value representing caffeine nutrient.
CALCIUM	Value representing calcium nutrient.
CHLORIDE	Value representing chloride nutrient.
CARBOHYDRATES	Value representing carbohydrates nutrient.
CHOLESTEROL	Value representing cholesterol nutrient.
CHROMIUM	Value representing chromium nutrient.
COPPER	Value representing copper nutrient.
DIETARY_FIBER	Value representing dietary fiber nutrient.
FOLIC_ACID	Value representing folic acid nutrient.
IODINE	Value representing iodine nutrient.
IRON	Value representing iron nutrient.
MAGNESIUM	Value representing magnesium nutrient.
MANGANESE	Value representing manganese nutrient.
MOLYBDENUM	Value representing molybdenum nutrient.
MONOUNSATURATED_FAT	Value representing monounsaturated fat nutrient.
NIACIN	Value representing niacin nutrient.
PANTOTHENIC_ACID	Value representing pantothenic acid nutrient.
PHOSPHORUS	Value representing phosphorus nutrient.

`POLYUNSATURATED_FAT`	Value representing polyunsaturated fat nutrient.
`POTASSIUM`	Value representing potassium nutrient.
`PROTEIN`	Value representing protein nutrient.
`RIBOFLAVIN`	Value representing riboflavin nutrient.
`SATURATED_FAT`	Value representing saturated fat nutrient.
`SELENIUM`	Value representing selenium nutrient.
`SODIUM`	Value representing sodium nutrient.
`SUGAR`	Value representing sugar nutrient.
`THIAMIN`	Value representing thiamin nutrient.
`TRANS_FAT`	Value representing trans fat nutrient.
`UNSATURATED_FAT`	Value representing unsaturated fat nutrient.
`VITAMIN_A`	Value representing vitamin A nutrient.
`VITAMIN_B12`	Value representing vitamin B12 nutrient.
`VITAMIN_B6`	Value representing vitamin B6 nutrient.
`VITAMIN_C`	Value representing vitamin C nutrient.
`VITAMIN_D`	Value representing vitamin D nutrient.
`VITAMIN_E`	Value representing vitamin E nutrient.
`VITAMIN_K`	Value representing vitamin K nutrient.
`ZINC`	Value representing zinc nutrient.
`FOLATE`	Value representing folate nutrient.

EnergyQuantity

Represents the energy quantity.

JSON representation

```
{
  "userProvidedUnit": enum (EnergyUnit (/health/reference/rest/v4/users.dataTypes.da
  "kcal": number
}
```

Fields

`userProvidedUnit`	`enum (EnergyUnit (/health/reference/rest/v4/users.dataTypes.dataPoints#EnergyUnit))` Optional. Value representing the user provided unit.
`kcal`	`number` Required. Value representing the energy in kilocalories.

EnergyUnit

Enum representing the unit of energy.

Enums

`ENERGY_UNIT_UNSPECIFIED`	Unspecified energy unit.
`JOULE`	Value representing joule.
`KILOJOULE`	Value representing kilojoule.
`KILOCALORIE`	Value representing kilocalorie.
`SMALL_CALORIE`	Value representing small calorie.
`CALORIE`	Value representing calorie.

MealType

Enum representing the meal type.

Enums

`MEAL_TYPE_UNSPECIFIED`	Unspecified meal type.
`BEFORE_BREAKFAST`	Value representing a meal before breakfast.
`BREAKFAST`	Value representing a breakfast.
`BEFORE_LUNCH`	Value representing a morning snack.
`LUNCH`	Value representing a lunch.
`BEFORE_DINNER`	Value representing an afternoon snack.
`DINNER`	Value representing dinner.
`AFTER_DINNER`	Value representing an evening snack.
`SNACK`	Value representing any meal outside of the usual three meals per day.
`ANYTIME`	Value representing any time (legacy NA).

NutritionLog.Serving

Represents different properties and information about the serving of a specific food.

JSON representation

```
{
  "foodMeasurementUnit": string,
  "foodMeasurementUnitDisplayName": string,
  "amount": number
}
```

Fields

`foodMeasurementUnit`	`string` Required. Food measurement unit
`foodMeasurementUnitDisplayName`	`string` Output only. Legacy measurement unit for serving size in singular form

	(e.g. "piece", "gram").
`amount`	`number` Optional. Amount of food consumed, fractional values are supported.

IrregularRhythmNotification

Represents an Irregular Rhythm Notification alert, indicating a potential sign of atrial fibrillation (AFib). This data type is based on SaMD feature and any changes to it may require additional review.

JSON representation

```
{
  "interval": {
    object (SessionTimeInterval (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "alertWindows": [
    {
      object (IrregularRhythmNotification.AlertWindow (/health/reference/rest/v4
    }
  ],
  "medicalDeviceInfo": {
    object (MedicalDeviceInfo (/health/reference/rest/v4/users.dataTypes.dataPoints#M
  }
}
```

Fields

`interval`	`object (SessionTimeInterval` (/health/reference/rest/v4/users.dataTypes.dataPoints#SessionTimeInterval) `)` Required. Observed interval.
`alertWindows[]`	`object (IrregularRhythmNotification.AlertWindow` (/health/reference/rest/v4/users.dataTypes.dataPoints#IrregularRhythmNotification.AlertWindow) `)`

	Optional. The overlapping analysis windows that were used to evaluate rhythm for potential AFib, containing specific information about the user's heart rhythm.
`medicalDeviceInfo`	**`object (`**[**`MedicalDeviceInfo`**](/health/reference/rest/v4/users.dataTypes.dataPoints#MedicalDeviceInfo)**`)`** Output only. The meta information for the compatible device used to conduct the measurement. Irregular Rhythm Notification measurements typically populate **`algorithmVersion`**, **`serviceVersion`**, and **`deviceModel`**.

IrregularRhythmNotification.AlertWindow

An analysis window evaluated for AFib.

Note: The current version of the algorithm will only produce alerts if all windows are positive. So anything returned from the API will always have the positive bit set to true. Internally, windows can be negative, however. We never save "inconclusive" windows (they aren't produced by the algorithm).

JSON representation

```
{
  "startTime": string,
  "startUtcOffset": string,
  "endTime": string,
  "endUtcOffset": string,
  "civilStartTime": {
    object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDa
  },
  "civilEndTime": {
    object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDa
  },
  "positive": boolean,
  "heartBeats": [
    {
      object (IrregularRhythmNotification.HeartBeat (/health/reference/rest/v4/u
    }
```

```
  ]
}
```

Fields

`startTime`	`string (Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) format)` Required. Observed interval. The start time of the analysis window. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`startUtcOffset`	`string (Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) format)` Required. The UTC offset of the user's timezone when the analysis window started. A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`endTime`	`string (Timestamp (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) format)` Required. The end time of the analysis window. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`endUtcOffset`	`string (Duration (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) format)` Required. The UTC offset of the user's timezone when the analysis window ended. A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".

`civilStartTime`	`object (`CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDateTime)`)` Output only. Observed interval start time in civil time in the timezone the subject is in at the start of the observed interval
`civilEndTime`	`object (`CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDateTime)`)` Output only. Observed interval end time in civil time in the timezone the subject is in at the end of the observed interval
`positive`	`boolean` Optional. Flag indicating whether the window was positive for AFib or not. A `true` value indicates that AFib was detected in this window. A `false` value means AFib was not detected, but does not guarantee the absence of AFib.
`heartBeats[]`	`object (`IrregularRhythmNotification.HeartBeat (/health/reference/rest/v4/users.dataTypes.dataPoints#IrregularRhythmNotification.HeartBeat)`)` Optional. All heart beats in the interval contained in this analysis window.

IrregularRhythmNotification.HeartBeat

A single heart beat measurement.

JSON representation

```
{
  "physicalTime": string,
  "utcOffset": string,
  "civilTime": {
    object (CivilDateTime (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDa
  },
  "beatsPerMinute": integer
}
```

Fields

`physicalTime`	`string (Timestamp` (https://protobuf.dev/reference/protobuf/google.protobuf/#timestamp) `format)` Required. The time of the heart beat measurement. Uses RFC 3339, where generated output will always be Z-normalized and use 0, 3, 6 or 9 fractional digits. Offsets other than "Z" are also accepted. Examples: "`2014-10-02T15:01:23Z`", "`2014-10-02T15:01:23.045123456Z`" or "`2014-10-02T15:01:23+05:30`".
`utcOffset`	`string (Duration` (https://protobuf.dev/reference/protobuf/google.protobuf/#duration) `format)` Required. The UTC offset of the user's timezone when the heart beat measurement occurred. A duration in seconds with up to nine fractional digits, ending with '`s`'. Example: "`3.5s`".
`civilTime`	`object (CivilDateTime` (/health/reference/rest/v4/users.dataTypes.dataPoints#CivilDateTime)`)` Output only. The civil time in the timezone the subject is in at the time of the observation.
`beatsPerMinute`	`integer` Required. The beats-per-minute value extrapolated from the time before the following heart beat. This is calculated as 60000 / rr, where rr is the gap between heart beats in milliseconds (IBI - Interbeat Interval).

MedicalDeviceInfo

Software as Medical Device (SaMD) metadata. Used to construct the Unique Device Identifier (UDI).

JSON representation

```
{
  "algorithmVersion": string,
  "serviceVersion": string,
  "firmwareVersion": string,
```

```
  "featureVersion": string,
  "deviceModel": string
}
```

Fields

`algorithmVersion`	`string` Output only. The algorithm version used by the feature.
`serviceVersion`	`string` Output only. The service version used by the feature.
`firmwareVersion`	`string` Output only. The firmware version running on the compatible device used to collect the data.
`featureVersion`	`string` Output only. The version of the feature/app running on the device.
`deviceModel`	`string` Output only. The model name or device type of the compatible device used to collect the data.

Electrocardiogram

Represents an Electrocardiogram (ECG) measurement session. This data type is based on SaMD feature and any changes to it may require additional review.

JSON representation

```
{
  "interval": {
    object (SessionTimeInterval (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "resultClassification": enum (Electrocardiogram.ResultClassification (/h
  "waveformSamples": [
    integer
  ],
```

```
  "medicalDeviceInfo": {
    object (MedicalDeviceInfo (/health/reference/rest/v4/users.dataTypes.dataPoints#M
  },
  "beatsPerMinuteAvg": string,
  "samplingFrequencyHertz": integer,
  "millivoltsScalingFactor": integer,
  "leadNumber": integer
}
```

Fields

interval	**object (SessionTimeInterval** (/health/reference/rest/v4/users.dataTypes.dataPoints#SessionTimeInterval) **)** Required. Observed interval. NOTE: Historical ECG data lacks timezone offsets, so **startUtcOffset** and **endUtcOffset** will be missing or default to zero. As a result, the civil time fields within this interval will default to UTC. It is recommended to use physical time fields instead for accurate time referencing. NOTE: The **startTime** and **endTime** of the interval are equal, representing the reading time.
resultClassification	**enum (Electrocardiogram.ResultClassification** (/health/reference/rest/v4/users.dataTypes.dataPoints#Electrocardiogram.ResultClassification) **)** Optional. The result classification of the ECG reading.
waveformSamples[]	**integer** Optional. An array of voltage values representing lead I ECG values. Each sample represents voltage difference in ECG graph. The first value in array corresponds to the start of the reading.
medicalDeviceInfo	**object (MedicalDeviceInfo** (/health/reference/rest/v4/users.dataTypes.dataPoints#MedicalDeviceInfo) **)** Output only. The meta information for the compatible device used to conduct the measurement.

	ECG measurements typically populate `firmwareVersion`, `featureVersion`, and `deviceModel`.
`beatsPerMinuteAvg`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Optional. Average heart rate recorded during ECG reading in beats per minute.
`samplingFrequencyHertz`	`integer` Optional. The sampling frequency of waveform samples in hertz.
`millivoltsScalingFactor`	`integer` Optional. The factor by which to divide waveform samples to get voltage in millivolts: millivolts = waveform_sample / millivoltsScalingFactor.
`leadNumber`	`integer` Optional. The number of leads used for ECG reading.

Electrocardiogram.ResultClassification

The classification of the ECG reading rhythm.

Enums	
`RESULT_CLASSIFICATION_UNSPECIFIED`	Unspecified result classification.
`NORMAL_SINUS_RHYTHM`	Heart rhythm appears normal. Corresponds to result "Normal Sinus Rhythm".
`ATRIAL_FIBRILLATION`	Signs of Atrial Fibrillation detected. Corresponds to result "Atrial Fibrillation".
`INCONCLUSIVE`	The reading is inconclusive as it could not be classified. Corresponds to result "Inconclusive".
`INCONCLUSIVE_HIGH_HEART_RATE`	The reading is inconclusive as it could not be classified because heart rate is high (>120bpm). Corresponds to result "Inconclusive: High heart rate".

`INCONCLUSIVE_LOW_HEART_RATE`	The reading is inconclusive as it could not be classified because heart rate is low (<50bpm). Corresponds to result "Inconclusive: Low heart rate".
`UNREADABLE`	The reading is unreadable.
`NOT_ANALYZED`	The reading was not analyzed.

DailyHeartRateZones

User's heart rate zone thresholds based on the Karvonen algorithm for a specific day.

JSON representation

```
{
  "date": {
    object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))
  },
  "heartRateZones": [
    {
      object (DailyHeartRateZones.HeartRateZone (/health/reference/rest/v4/users.d
    }
  ]
}
```

Fields

`date`	`object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))` Required. Date (in user's timezone) of the heart rate zones record.
`heartRateZones[]`	`object (DailyHeartRateZones.HeartRateZone (/health/reference/rest/v4/users.dataTypes.dataPoints#DailyHeartRateZones.HeartRateZone))` Required. The heart rate zones.

DailyHeartRateZones.HeartRateZone

The heart rate zone.

JSON representation

```
{
  "heartRateZoneType": enum (HeartRateZoneType (/health/reference/rest/v4/users.da
  "minBeatsPerMinute": string,
  "maxBeatsPerMinute": string
}
```

Fields

`heartRateZoneType`	`enum (`HeartRateZoneType (/health/reference/rest/v4/users.dataTypes.dataPoints#HeartRateZoneType)`)` Required. The heart rate zone type.
`minBeatsPerMinute`	`string (`int64 (https://developers.google.com/discovery/v1/type-format) `format)` Required. Minimum heart rate for this zone in beats per minute.
`maxBeatsPerMinute`	`string (`int64 (https://developers.google.com/discovery/v1/type-format) `format)` Required. Maximum heart rate for this zone in beats per minute.

HeartRateZoneType

The heart rate zone type.

Enums

`HEART_RATE_ZONE_TYPE_UNSPECIFIED`	Unspecified heart rate zone.
`LIGHT`	The light heart rate zone.
`MODERATE`	The moderate heart rate zone.

VIGOROUS	The vigorous heart rate zone.
PEAK	The peak heart rate zone.

HydrationLog

Holds information about a user logged hydration.

JSON representation

```
{
  "interval": {
    object (SessionTimeInterval (/health/reference/rest/v4/users.dataTypes.dataPoints
  },
  "amountConsumed": {
    object (VolumeQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Volum
  }
}
```

Fields

interval	**object (SessionTimeInterval** (/health/reference/rest/v4/users.dataTypes.dataPoints#SessionTimeInterval) **)** Required. Observed interval.
amountConsumed	**object (VolumeQuantity** (/health/reference/rest/v4/users.dataTypes.dataPoints#VolumeQuantity) **)** **)** Required. Amount of liquid (ex. water) consumed.

VolumeQuantity

Represents the volume quantity.

JSON representation

```
{
  "userProvidedUnit": enum (VolumeUnit (/health/reference/rest/v4/users.dataTypes.da
  "milliliters": number
}
```

Fields

`userProvidedUnit`	`enum (VolumeUnit (/health/reference/rest/v4/users.dataTypes.dataPoints#VolumeUnit))` Optional. Value representing the user provided unit, used only for user-facing input and display purposes. In the API format, all volume quantities are converted to milliliters.
`milliliters`	`number` Required. Value representing the volume in milliliters.

VolumeUnit

Enum representing the unit of volume.

Enums

`VOLUME_UNIT_UNSPECIFIED`	Unspecified volume unit.
`CUP_IMPERIAL`	Cup (imperial)
`CUP_US`	Cup (US)
`FLUID_OUNCE_IMPERIAL`	Fluid ounce (imperial)
`FLUID_OUNCE_US`	Fluid ounce (US)
`LITER`	Liter
`MILLILITER`	Milliliter
`PINT_IMPERIAL`	Pint (imperial)

`PINT_US`	Pint (US)

Food

Represents a food item.

JSON representation

```
{
  "displayName": string,
  "brand": string,
  "accessLevel": enum (FoodAccessLevel (/health/reference/rest/v4/users.dataTypes.da
  "description": string,
  "languageCode": string,
  "mealType": enum (MealType (/health/reference/rest/v4/users.dataTypes.dataPoints#Me
  "nutrients": [
    {
      object (NutrientQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#
    }
  ],
  "energyFromFat": {
    object (EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Energ
  },
  "totalCarbohydrate": {
    object (WeightQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Weigh
  },
  "totalFat": {
    object (WeightQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Weigh
  },
  "energyMin": {
    object (EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Energ
  },
  "energyAvg": {
    object (EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Energ
  },
  "energyMax": {
    object (EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#Energ
  },
  "defaultServing": {
    object (Food.FoodServing (/health/reference/rest/v4/users.dataTypes.dataPoints#Foo
  },
  "servings": [
```

```
    {
      object (Food.FoodServing (/health/reference/rest/v4/users.dataTypes.dataPoints#
    }
  ]
}
```

Fields

displayName	**string** Required. The display name of the food.
brand	**string** Optional. The brand of the food.
accessLevel	**enum (FoodAccessLevel** (/health/reference/rest/v4/users.dataTypes.dataPoints#FoodAccessLevel)) Required. The access level of the food.
description	**string** Optional. The description of the food.
languageCode	**string** Optional. The language code where the food is available in format xx-XX. Supported values are defined in **Settings.food_language_code** (/health/reference/rest/v4/Settings#FIELDS.food_language_code).
mealType	**enum (MealType** (/health/reference/rest/v4/users.dataTypes.dataPoints#MealType)) Optional. The meal type associated with this food.
nutrients[]	**object (NutrientQuantity** (/health/reference/rest/v4/users.dataTypes.dataPoints#NutrientQuantity)) Optional. Value representing the nutrients of the food for the default serving.
energyFromFat	**object (EnergyQuantity**

	(/health/reference/rest/v4/users.dataTypes.dataPoints#EnergyQuantity)) Optional. Value representing the energy from fat of the food for the default serving.
`totalCarbohydrate`	`object (`WeightQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#WeightQuantity)) Optional. Value representing the total carbohydrate of the food for the default serving.
`totalFat`	`object (`WeightQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#WeightQuantity)) Optional. Value representing the total fat of the food for the default serving.
`energyMin`	`object (`EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#EnergyQuantity)) Optional. Value representing the minimum energy of the food for the default serving.
`energyAvg`	`object (`EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#EnergyQuantity)) Optional. Value representing the average energy of the food for the default serving.
`energyMax`	`object (`EnergyQuantity (/health/reference/rest/v4/users.dataTypes.dataPoints#EnergyQuantity)) Optional. Value representing the maximum energy of the food for the default serving.
`defaultServing`	`object (`Food.FoodServing (/health/reference/rest/v4/users.dataTypes.dataPoints#Food.FoodServing)) Required. Value representing the default serving of the food.

`servings[]`	`object (`Food.FoodServing (/health/reference/rest/v4/users.dataTypes.dataPoints#Food.FoodServing)`)` Optional. The serving of the food.

FoodAccessLevel

Enum representing the access level of a food item.

Enums

`FOOD_ACCESS_LEVEL_UNSPECIFIED`	Unspecified food access level.
`FOOD_ACCESS_LEVEL_PUBLIC`	Public food access level.
`FOOD_ACCESS_LEVEL_PRIVATE`	Private food access level.

Food.FoodServing

Represents different properties and information about the serving of a specific food.

JSON representation

```
{
  "foodMeasurementUnit": string,
  "foodMeasurementUnitDisplayName": string,
  "foodMeasurementUnitDisplayNamePlural": string,
  "multiplier": number,
  "amount": number
}
```

Fields

`foodMeasurement`	`string`

`Unit`	Required. Food measurement unit
`foodMeasurementUnitDisplayName`	`string` Output only. Legacy measurement unit for serving size in singular form (e.g. "piece", "gram").
`foodMeasurementUnitDisplayNamePlural`	`string` Output only. Legacy measurement unit for serving size in plural form (e.g. "pieces", "grams").
`multiplier`	`number` Optional. Value representing the multiplier used to compute the energy when using this serving instead of the default serving.
`amount`	`number` Optional. Amount of food consumed, fractional values are supported.

TimeInHeartRateZone

Time in heart rate zone record. It's an interval spent in specific heart rate zone.

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
  "heartRateZoneType": enum (HeartRateZoneType (/health/reference/rest/v4/users.da
}
```

Fields

`interval`	`object (ObservationTimeInterval` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval)) Required. Observed interval.

`heartRateZoneType`	`enum (`HeartRateZoneType (/health/reference/rest/v4/users.dataTypes.dataPoints#HeartRateZoneType)`)` Required. Heart rate zone type.

ActiveMinutes

Record of active minutes in a given time interval.

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
  "activeMinutesByActivityLevel": [
    {
      object (ActiveMinutes.ActiveMinutesByActivityLevel (/health/reference/res
    }
  ]
}
```

Fields

`interval`	`object (`ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval)`)` Required. Observed interval.
`activeMinutesByActivityLevel[]`	`object (`ActiveMinutes.ActiveMinutesByActivityLevel (/health/reference/rest/v4/users.dataTypes.dataPoints#ActiveMinutes.ActiveMinutesByActivityLevel)`)` Required. Active minutes by activity level. At most one record per activity level is allowed.

ActiveMinutes.ActiveMinutesByActivityLevel

Active minutes at a given activity level.

JSON representation

```
{
  "activityLevel": enum (ActiveMinutes.ActivityLevel (/health/reference/rest/v4/
  "activeMinutes": string
}
```

Fields

`activityLevel`	`enum (ActiveMinutes.ActivityLevel` (/health/reference/rest/v4/users.dataTypes.dataPoints#ActiveMinutes.ActivityLevel) `)` Required. The level of activity.
`activeMinutes`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Required. Number of whole minutes spent in activity.

ActiveMinutes.ActivityLevel

Activity level.

Enums

`ACTIVITY_LEVEL_UNSPECIFIED`	Activity level is unspecified.
`LIGHT`	Light activity level.
`MODERATE`	Moderate activity level.
`VIGOROUS`	Vigorous activity level.

RespiratoryRateSleepSummary

Records respiratory rate details during sleep. Can have multiple per day if the user sleeps multiple times.

JSON representation


```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "deepSleepStats": {
    object (RespiratoryRateSleepSummary.RespiratoryRateSleepSummaryStatis
  },
  "lightSleepStats": {
    object (RespiratoryRateSleepSummary.RespiratoryRateSleepSummaryStatis
  },
  "remSleepStats": {
    object (RespiratoryRateSleepSummary.RespiratoryRateSleepSummaryStatis
  },
  "fullSleepStats": {
    object (RespiratoryRateSleepSummary.RespiratoryRateSleepSummaryStatis
  }
}
```


Fields

sampleTime	**object (ObservationSampleTime** (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime)) Required. The time at which respiratory rate was measured.
deepSleepStats	**object** (RespiratoryRateSleepSummary.RespiratoryRateSleepSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#RespiratoryRateSleepSummary.RespiratoryRateSleepSummaryStatistics)) Optional. Respiratory rate statistics for deep sleep.
lightSleepStats	**object**

	(RespiratoryRateSleepSummary.RespiratoryRateSleepSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#RespiratoryRateSleepSummary.RespiratoryRateSleepSummaryStatistics)) Optional. Respiratory rate statistics for light sleep.
`remSleepStats`	`object` (RespiratoryRateSleepSummary.RespiratoryRateSleepSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#RespiratoryRateSleepSummary.RespiratoryRateSleepSummaryStatistics)) Optional. Respiratory rate statistics for REM sleep.
`fullSleepStats`	`object` (RespiratoryRateSleepSummary.RespiratoryRateSleepSummary (/health/reference/rest/v4/users.dataTypes.dataPoints#RespiratoryRateSleepSummary.RespiratoryRateSleepSummaryStatistics)) Required. Full respiratory rate statistics.

RespiratoryRateSleepSummary.RespiratoryRateSleepSummaryStatistics

Respiratory rate statistics for a given sleep stage.

JSON representation

```
{
  "breathsPerMinute": number,
  "standardDeviation": number,
  "signalToNoise": number
}
```

Fields

`breathsPerMinute`	`number`

	Required. Average breaths per minute.
`standardDeviation`	`number` Optional. Standard deviation of the respiratory rate during sleep.
`signalToNoise`	`number` Optional. How trustworthy the data is for the computation.

DailyRespiratoryRate

A daily average respiratory rate (breaths per minute) for a day of the year. One data point per day calculated for the main sleep.

JSON representation

```
{
  "date": {
    object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))
  },
  "breathsPerMinute": number
}
```

Fields

`date`	`object (Date (/health/reference/rest/v4/users.dataTypes.dataPoints#Date))` Required. The date on which the respiratory rate was measured.
`breathsPerMinute`	`number` Required. The average number of breaths taken per minute.

SwimLengthsData

Swim lengths data over the time interval.

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
  "swimStrokeType": enum (SwimLengthsData.SwimStrokeType (/health/reference/res
  "strokeCount": string
}
```

Fields

interval	**object (ObservationTimeInterval** (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval) **)** Required. Observed interval.
swimStrokeType	**enum (SwimLengthsData.SwimStrokeType** (/health/reference/rest/v4/users.dataTypes.dataPoints#SwimLengthsData.SwimStrokeType) **)** Required. Swim stroke type.
strokeCount	**string (int64** (https://developers.google.com/discovery/v1/type-format) **format)** Required. Number of strokes in the lap.

SwimLengthsData.SwimStrokeType

Swim stroke type.

Enums

SWIM_STROKE_TYPE_UNSPECIFIED	Swim stroke type is unspecified.
FREESTYLE	Freestyle swim stroke type.
BACKSTROKE	Backstroke swim stroke type.

BREASTSTROKE	Breaststroke swim stroke type.
BUTTERFLY	Butterfly swim stroke type.

Height

Body height measurement.

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "heightMillimeters": string
}
```

Fields

`sampleTime`	`object (ObservationSampleTime` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime) `)` Required. The time at which the height was recorded.
`heightMillimeters`	`string (int64` (https://developers.google.com/discovery/v1/type-format) `format)` Required. Height of the user in millimeters.

CoreBodyTemperature

Core body temperature measurement, distinct from peripheral body temperature, reflects the temperature of the body's internal organs.

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "measurementLocation": enum (CoreBodyTemperature.MeasurementLocation (/h
  "id": string,
  "temperatureCelsius": number
}
```

Fields

sampleTime	**object (ObservationSampleTime** (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime)) Required. The time at which core body temperature was measured.
measurementLocation	**enum (CoreBodyTemperature.MeasurementLocation** (/health/reference/rest/v4/users.dataTypes.dataPoints#CoreBodyTemperature.MeasurementLocation)) Optional. The location of the core body temperature measurement.
id	**string** Optional. The unique identifier of the core body temperature measurement.
temperatureCelsius	**number** Required. The core body temperature in Celsius.

CoreBodyTemperature.MeasurementLocation

Measurement location for core body temperature.

Enums

MEASUREMENT_LOCATION_	Measurement location is unspecified.

UNSPECIFIED	
OTHER	Other measurement location.
ARMPIT	Armpit measurement location.
BODY	Body measurement location.
EAR	Ear measurement location.
FINGER	Finger measurement location.
GASTRO_INTESTINAL	Gastro-intestinal measurement location.
MOUTH	Mouth measurement location.
RECTUM	Rectum measurement location.
TOE	Toe measurement location.
EAR_DRUM	Ear drum measurement location.
TEMPORAL_ARTERY	Temporal artery measurement location.
FOREHEAD	Forehead measurement location.
URINARY_BLADDER	Urinary bladder measurement location.
NASAL	Nasal measurement location.
NASOPHARYNGEAL	Nasopharyngeal measurement location.
WRIST	Wrist measurement location.
VAGINA	Vagina measurement location.

ActiveEnergyBurned

Energy burned as part of an activity, excluding the basal energy burn.

JSON representation

```
{
  "interval": {
    object (ObservationTimeInterval (/health/reference/rest/v4/users.dataTypes.dataP
  },
```

```
  "kcal": number
}
```

Fields

`interval`	`object (`[ObservationTimeInterval](/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationTimeInterval)`)` Required. Observed interval
`kcal`	`number` Required. Energy burned during an activity, measured in kilocalories.

FoodMeasurementUnit

Represents a food measurement unit.

JSON representation

```
{
  "displayName": string,
  "pluralDisplayName": string
}
```

Fields

`displayName`	`string` Required. The display name of the food measurement unit (e.g., "gram", "piece").
`pluralDisplayName`	`string` Optional. The plural display name of the food measurement unit (e.g., "grams", "pieces").

BloodGlucose

Represents a blood glucose level measurement. LINT: LEGACY_NAMES

JSON representation

```
{
  "sampleTime": {
    object (ObservationSampleTime (/health/reference/rest/v4/users.dataTypes.dataPoir
  },
  "measurementSource": enum (BloodGlucose.MeasurementSource (/health/reference
  "mealType": enum (BloodGlucose.MealType (/health/reference/rest/v4/users.dataType
  "measurementTiming": enum (BloodGlucose.MeasurementTiming (/health/reference
  "specimen": enum (BloodGlucose.Specimen (/health/reference/rest/v4/users.dataType
  "notes": string,
  "bloodGlucoseMilligramsPerDeciliter": number
}
```

Fields

`sampleTime`	`object (ObservationSampleTime` (/health/reference/rest/v4/users.dataTypes.dataPoints#ObservationSampleTime) `)` Required. The time at which blood glucose was measured.
`measurementSource`	`enum (BloodGlucose.MeasurementSource` (/health/reference/rest/v4/users.dataTypes.dataPoints#BloodGlucose.MeasurementSource) `)` Optional. Source of the measurement.
`mealType`	`enum (BloodGlucose.MealType` (/health/reference/rest/v4/users.dataTypes.dataPoints#BloodGlucose.MealType) `)` Optional. Meal type of the measurement.
`measurementTiming`	`enum (BloodGlucose.MeasurementTiming` (/health/reference/rest/v4/users.dataTypes.dataPoints#BloodGlucose.MeasurementTiming)

	) Optional. Timing of the measurement.
`specimen`	`enum (`BloodGlucose.Specimen (/health/reference/rest/v4/users.dataTypes.dataPoints#BloodGlucose.Specimen))` Optional. Type of body fluid used to measure the blood glucose.
`notes`	`string` Optional. Standard free-form notes captured at manual logging.
`bloodGlucoseMilligramsPerDeciliter`	`number` Required. Blood glucose level concentration in mg/dL.

BloodGlucose.MeasurementSource

The clinical method or tool used to measure the blood glucose level.

Enums	
`MEASUREMENT_SOURCE_UNSPECIFIED`	Unspecified measurement source.
`SELF_MONITORING_BLOOD_GLUCOSE`	Self-monitoring of blood glucose (Blood glucose meter)
`CONTINUOUS_GLUCOSE_MONITORING`	Continuous glucose monitoring device
`LAB_TEST`	Laboratory test

BloodGlucose.MealType

Meal type associated with the measurement.

Enums

`MEAL_TYPE_UNSPECIFIED`	Unspecified meal type.
`BREAKFAST`	Breakfast.
`LUNCH`	Lunch.
`DINNER`	Dinner.
`SNACK`	Snack.

BloodGlucose.MeasurementTiming

Timing of the measurement.

Enums	
`MEASUREMENT_TIMING_UNSPECIFIED`	Unspecified measurement timing.
`AFTER_MEAL`	Measurement taken after meal.
`BEFORE_MEAL`	Measurement taken before meal.
`FASTING`	Measurement taken while fasting.
`GENERAL`	General measurement (not associated with a meal or time of day).
`BEFORE_BED`	Measurement taken before bed.
`OVER_NIGHT`	Measurement taken over night.

BloodGlucose.Specimen

Type of body fluid used to measure the blood glucose.

Enums	
`SPECIMEN_`	Unspecified specimen.

`UNSPECIFIED`	
`CAPILLARY_BLOOD`	Capillary blood.
`INTERSTITIAL_FLUID`	Interstitial fluid.
`PLASMA`	Plasma.
`SERUM`	Serum.
`TEARS`	Tears.
`WHOLE_BLOOD`	Whole blood.

DataSource

Data Source definition to track the origin of data.

Each health data point, regardless of the complexity or data model (whether a simple step count or a detailed sleep session) must retain information about its source of origin (e.g. the device or app that collected it).

JSON representation

```
{
  "recordingMethod": enum (DataSource.RecordingMethod (/health/reference/rest/v4
  "device": {
    object (DataSource.Device (/health/reference/rest/v4/users.dataTypes.dataPoints#D
  },
  "application": {
    object (DataSource.Application (/health/reference/rest/v4/users.dataTypes.dataPo
  },
  "platform": enum (DataSource.Platform (/health/reference/rest/v4/users.dataTypes.
}
```

Fields

`recordingMethod`	`enum (`DataSource.RecordingMethod (/health/reference/rest/v4/users.dataTypes.dataPoints#DataSource.RecordingMethod)`)`

	Optional. Captures how the data was recorded.
`device`	`object (`DataSource.Device (/health/reference/rest/v4/users.dataTypes.dataPoints#DataSource.Device)`)` Optional. Captures metadata for raw data points originating from devices. We expect this data source to be used for data points written on device sync.
`application`	`object (`DataSource.Application (/health/reference/rest/v4/users.dataTypes.dataPoints#DataSource.Application)`)` Output only. Captures metadata for the application that provided this data.
`platform`	`enum (`DataSource.Platform (/health/reference/rest/v4/users.dataTypes.dataPoints#DataSource.Platform)`)` Output only. Captures the platform that uploaded the data.

DataSource.RecordingMethod

The method by which the data was recorded.

Enums	
`RECORDING_METHOD_UNSPECIFIED`	The recording method is unspecified.
`MANUAL`	The data was manually entered by the user.
`PASSIVELY_MEASURED`	The data was passively measured by a device.
`DERIVED`	The data was derived from other data, e.g., by an algorithm in the backend.
`ACTIVELY_MEASURED`	The data was actively measured by a device.

UNKNOWN	The recording method is unknown. This is set when the data is uploaded from a third party app that does not provide this information.

DataSource.Device

Captures metadata about the device that recorded the measurement.

JSON representation

```
{
  "formFactor": enum (DataSource.Device.FormFactor (/health/reference/rest/v4/us
  "manufacturer": string,
  "displayName": string
}
```

Fields

formFactor	**enum (DataSource.Device.FormFactor** (/health/reference/rest/v4/users.dataTypes.dataPoints#DataSource.Device.FormFactor)) Optional. Captures the form factor of the device.
manufacturer	**string** Optional. An optional manufacturer of the device.
displayName	**string** Optional. An optional name for the device.

DataSource.Device.FormFactor

Form factor of the device, e.g. phone, watch, band, etc.

Enums

FORM_FACTOR_	The form factor is unspecified.

UNSPECIFIED	
FITNESS_BAND	The device is a fitness band.
WATCH	The device is a watch.
PHONE	The device is a phone.
RING	The device is a ring.
CHEST_STRAP	The device is a chest strap.
SCALE	The device is a scale.
TABLET	The device is a tablet.
HEAD_MOUNTED	The device is a head mounted device.
SMART_DISPLAY	The device is a smart display.

DataSource.Application

Optional metadata for the application that provided this data.

JSON representation

```
{
  "packageName": string,
  "webClientId": string,
  "googleWebClientId": string
}
```

Fields

packageName	**string** Output only. A unique identifier for the mobile application that was the source of the data. This is typically the application's package name on Android (e.g., **com.google.fitbit**) or the bundle ID on iOS. This field is informational and helps trace data origin. This field is system-populated when the data is uploaded from the Fitbit mobile application, Health Connect or Health Kit.

webClientId	**string** Output only. The client ID of the application that recorded the data. This ID is a legacy Fitbit API client ID, which is different from a Google OAuth client ID. Example format: **ABC123**. This field is system-populated and used for tracing data from legacy Fitbit API integrations. This field is system-populated when the data is uploaded from a legacy Fitbit API integration.
googleWebClientId	**string** Output only. The Google OAuth 2.0 client ID of the web application or service that recorded the data. This is the client ID used during the Google OAuth flow to obtain user credentials. This field is system-populated when the data is uploaded from Google Web API.

DataSource.Platform

The platform that uploaded the data. Additional values may be added in the future. Clients should be prepared to handle unknown values gracefully.

Enums	
PLATFORM_UNSPECIFIED	The platform is unspecified.
FITBIT	The data was uploaded from Fitbit.
HEALTH_CONNECT	The data was uploaded from Health Connect.
HEALTH_KIT	The data was uploaded from Health Kit.
FIT	The data was uploaded from Google Fit.
FITBIT_WEB_API	The data was uploaded from Fitbit legacy Web API.
NEST	The data was uploaded from Nest devices.
GOOGLE_WEB_API	The data was uploaded from Google Health API.
GOOGLE_PARTNER_	The data was uploaded from Google Partner Integrations.

INTEGRATION

Methods

batchDelete (/health/reference/rest/v4/users.dataTypes.dataPoints/batchDelete)	Delete a batch of identifyable data points.
create (/health/reference/rest/v4/users.dataTypes.dataPoints/create)	Creates a single identifiable data point.
dailyRollUp (/health/reference/rest/v4/users.dataTypes.dataPoints/dailyRollUp)	Roll up data points over civil time intervals for supported data types.
exportExerciseTcx (/health/reference/rest/v4/users.dataTypes.dataPoints/exportExerciseTcx)	Exports exercise data in TCX format.
get (/health/reference/rest/v4/users.dataTypes.dataPoints/get)	Get a single identifyable data point.
list (/health/reference/rest/v4/users.dataTypes.dataPoints/list)	Query user health and fitness data points.
patch (/health/reference/rest/v4/users.dataTypes.dataPoints/patch)	Updates a single identifiable data point.
reconcile (/health/reference/rest/v4/users.dataTypes.dataPoints/reconcile)	Reconcile data points from multiple data sources into a single data stream.
rollUp	Roll up data points over physical time intervals for supported data types.

(/health/reference/rest/v4/users.dataTypes.dataPoints/rollUp)

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-05-26 UTC.